

Revisiting Automatic Pipelining: Gate-level Forwarding and Speculation

Shuyao Cheng^{1,2,3}, Chongxiao Li^{1,2,3}, Zidong Du^{1,4}, Rui Zhang^{1,*}, Xing Hu^{1,4}, Xiaqing Li¹, Guanglin Xu¹, Yuanbo Wen¹, Qi Guo^{1,*}

chengshuyao19z, lichongxiao21s, duzidong, zhangrui, huxing, lixiaqing, xuguanglin, wenyuanbo, guoqi@ict.ac.cn

¹State Key Lab of Processors, Institute of Computing Technology, Chinese Academy of Sciences ²University of Chinese Academy of Sciences ³Cambricon Technologies ⁴Shanghai Innovation Center for Processor Technologies

ABSTRACT

Pipelining is a widely applied micro-architectural performance optimization and requires non-trivial designs for better execution throughput. The key to pipeline throughput optimization is to resolve data hazards caused by read-after-write (RAW) dependencies, which are traditionally tackled by forwarding and speculation to avoid pipeline stalls. However, existing approaches are conducted based on high-level dataflow analysis, with potential loss of optimization opportunities for lack of analysis of the netlist structures.

We propose an efficient method to resolve RAW dependencies with low-level netlist analysis by gate-level forwarding and speculation. With a greedy search method to detect and resolve short-delay gate-level signal paths for forwarding and an approximate circuit synthesis method with formal verification for gate-level speculation, the method efficiently utilizes the gate-level information to further improve pipeline throughput. Even if the gate-level design space is vast, by assembling the logic gates in the short-delay signal paths in fewer adjacent pipeline stages, the forwarding paths and the speculator bypass invalid pipeline stages and thus conceal the latency of these paths. We conduct experiments on the widely-used ISCAS/EPFL benchmark circuits and a large-scale RISC-V CPU, and our approach can find better designs than human experts.

1 INTRODUCTION

Pipelining is a crucial and widely adopted technique for improving the computational throughput of digital circuits [21, 25]. In a pipelined design, the datapath is decomposed into multiple stages connected with flip-flops, such that the execution of multi-cycle computations can be overlapped in time to hide the latency. While an ideal pipeline can complete one computation at each clock cycle in steady states, it may slow down when resolving potential read-after-write (RAW) data dependencies between computations. For minimizing pipeline stalls due to dependencies, pipeline interlock mechanisms involving stalling, data forwarding, and speculation have been developed for modern pipelined processor designs [26].

In this paper, we argue that conventional pipeline interlock mechanisms, designed in operator granularity, overlook opportunities for further reducing pipeline stalls. These include human design practices and automatic pipelining techniques [21]. In the operator

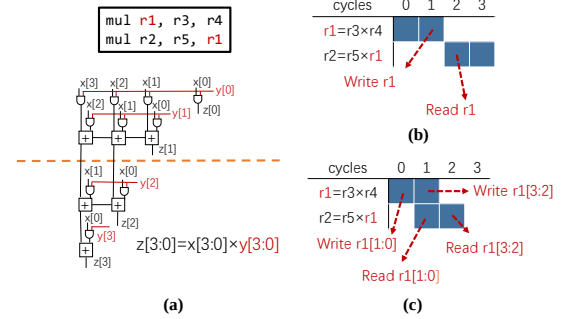


Figure 1: A simplified example of gate-level pipeline optimization (a) A 2-stage pipeline design for a 4-bit multiplier in processor, and the instructions with RAW dependency. (b) The execution flow of the high-level pipeline design. The pipeline has to stall, awaiting the multiplier calculates `r1`. (c) The execution flow of the gate-level pipeline design. Since the lower bits `r1[1:0]` are calculated earlier than `r1[3:2]` and are the only bits used in Stage 1, these signals can be forwarded earlier, eliminating the redundant pipeline stalls.

granularity, the availability of the processed data word is determined atomically, depending on that of the signal bit produced by the longest path [14, 18, 20]. In data forwarding, a feedback path for the whole data word is built between a pipeline stage producing data and the corresponding consumption stage [20]. In data speculation, the dependent data word is predicted with existing execution traces [8, 15, 23]. In short, conventional methods assume simultaneous arrivals of all signal bits of a data word from all gate-level signal paths, behaving too conservatively in calculating the required cycle of pipeline stalls, and thus cannot unearth the best performance of a circuit design.

We observe that there exist a number of redundant pipeline stalls in the conventional pipeline design based on high-level dataflow analysis, and these redundant stalls can be resolved with gate-level analysis. Figure 1 illustrates a simplified example of the gate-level throughput optimization, which includes a 4-bit multiplier of a CPU design in Figure 1(a), with a RAW dependency of two continuous instructions. Specifically, the orange dashed line is the pipeline stage boundary. The lower two output bits `z[1:0]` are calculated in the first stage from input signals `x[1:0]` and `y[1:0]`, and `z[3:2]` are calculated in the second stage with `x[3:0]` and `y[3:2]`. Since there is a RAW dependency between `z` and `y`, in the high-level analysis, all signals of the same data, e.g. `z` and `y`, have the same forwarding path and have to stall until all bits `y[3:0]` are calculated, as illustrated in Figure 1(b). However, in the gate-level analysis, output bits `z[1:0]`



can be forwarded to the input $y[1:0]$ earlier, and output bits $z[3:2]$ can be forwarded to the input $y[3:2]$ next; the pipeline does not need to stall. With gate-level forwarding, the netlist structure conceals the RAW latency and thus improves the pipeline throughput.

Even though the potential throughput improvement is clear, optimizing the pipeline with gate-level analysis is still challenging. The netlist, despite undergoing high-level dataflow analysis with dozens of nodes, can contain thousands of gates and signal paths, creating an extremely large exploration space. Such a large exploration space poses a significant barrier to the straightforward application of current pipeline design methodologies at the gate level. Concretely, for human experts, it is hardly possible to resolve gate-level hazards from netlists that lack readability. Different netlists for the same register-transfer level (RTL) code are optimized differently in the gate-level optimization, and thus high-level human knowledge is inefficient for further gate-level optimization. For automatic methods, the widely used rule-based deductive methods also rely on high-level human knowledge and cannot work well on the proposed optimization. For example, automatic pipelining methods at architectural level [3, 5, 9], High-level Synthesis [1, 6, 16, 19], and circuit level [10, 13, 22], can only optimize the circuit parts with feed-forward logic or analyze the RAW dependencies based on the high-level human-designed rules.

To this end, we propose an efficient gate-level pipeline throughput optimization method by gate-level forwarding and speculation, resolving RAW dependencies. The key insight is that in the gate-level analysis, one high-level datapath is separated into individual gate-level signal paths in the netlist, and the latency of some short-delay paths can be reduced. By assembling the logic gates in the short-delay signal paths in fewer adjacent pipeline stages, the forwarding paths bypass invalid pipeline stages and thus conceal the latency. For gate-level forwarding, the proposed method greedily assembles the logic gates in the short-delay paths to as late stage of the pipeline as possible, as well as gate-level forwarding path for short-delay paths to bypass the invalid early stages. For gate-level speculation, the proposed method approximately designs the speculator with Decision Trees with formal verification according to a software simulator containing all actual execution results.

The proposed method is implemented and evaluated on various circuits, including standard circuit benchmarks ISCAS85/EPFL and a RISC-V32 CPU design. Experimental results show that the approach can increase the pipeline throughput by $1.57\times$, outperforming state-of-the-art automatic pipelining approaches. More importantly, with the fine-grained gate-level design space, our approach can find better-pipelined designs than human designers by 31% on average.

In summary, the major contributions of this paper are:

- We uncover the benefit of gate-level forwarding and speculation for pipeline performance optimization in the presence of RAW dependencies.
- We propose an efficient automatic pipelining method for the fine-grained RAW analysis with gate-level forwarding and speculation, based on concretely resolving the latency of short-delay signal paths.
- We implement the methodology and evaluate it on standard circuit benchmarks and a RISC-V CPU core, outperforming

state-of-the-art automatic pipelining optimizations. More importantly, with the fine-grained design space, our approach can find better-pipelined designs than human designers.

2 RELATED WORKS

2.1 Automatic Pipelining

Automatic pipelining can significantly reduce human efforts to construct high-performance *sequential circuits*. It achieves this by generating pipelined circuits from a single-cycle input specification, such as C or RTL code, to enhance throughput. Its study and application span various stages of the circuit design process. TurboSYN is introduced to minimize the clock period of sequential circuits by retiming and pipelining [4]. For feed-forwarding circuits with no potential data hazards, automatic pipelining is applied to optimize the clock period constraint and has already been applied in the ASIC and FPGA design flow [13, 17, 22]. However, applying automatic pipelining to looped circuits poses more significant challenges because of the RAW hazards. Current pipelining methods optimize the throughput of the latency-insensitive circuits, also referred to as dataflow circuits or elastic systems, by asynchronous data forwarding based on dependency analysis. As the dependency analysis relies on human knowledge, whether through human analysis of the dataflow [5, 9] or manual rules for automatic code analysis [1, 6, 16, 27], they lack the inclusion of specific gate-level design structural details. Thus current automatic pipeline methods cannot explore the gate-level design space with more netlist information.

2.2 Program-by-Example Circuit Synthesis

Program-by-example (PBE) is an approach in inductive program synthesis where the goal is to construct programs that satisfy specifications based solely on input-output examples [11]. PBE has seen increasing applications in circuit synthesis to generate netlists meeting design requirements automatically. This combines techniques from electronic design automation and machine learning. Common PBE techniques for circuit synthesis mainly fall into four categories, i.e., decision trees, heuristic search, simulated annealing, and neural networks [2, 11, 12, 24]. Moreover, there have been recent EDA contests to show some PBE applications on the EDA cases. In IC-CAD contests 2019, Problem A is a PBE problem that shows the application in automating Engineering Change Order (ECO), logic synthesis, and model checking problems [12]. In the IWLS contest 2020, PBE problems exist for complex cases such as arithmetic circuits and even image detection [24]. However, existing PBE methods have limitations in achieving high enough accuracy or formal verification needed for correct-by-construction circuit synthesis. Ongoing research aims to improve PBE techniques to enable more robust and verifiable circuit synthesis from examples.

3 METHODOLOGY

3.1 Overview: Gate-level RAW Optimization

In conventional pipeline designs with high-level dataflow analysis, the RAW dependencies are analyzed without netlist structural information and thus have the potential for further optimization. We propose an efficient method to utilize the fine-grained information.

In the gate-level analysis, a single datapath can be separated into a group of gate-level signal paths in the netlist. For one specific input data, only part of these paths are valid. These signal

paths are various, with long-delay paths containing more gates and short-delay paths containing fewer, and the latency of short-delay signal paths can be further optimized for better pipeline throughput. Concretely, we formulate the gate-level pipeline throughput optimization. The average throughput can be calculated by cycle-per-inputs (CPI), corresponding to cycle-per-instruction (CPI) in the CPU design. It is fully pipelined for dataflow without any potential RAW dependency; thus, $CPI = 1$. For dataflow with potential RAW dependencies, the average CPI is defined by the RAW probability (proportion of cases with RAW dependencies and all possible cases, P_{data}) and RAW latency (cycles to obtain the required forwarded data, L_{data} the latency of the longest path). With the best high-level pipeline design, the throughput of high-level pipeline design is $CPI_{data} = P_{data} \times L_{data} + (1 - P_{data}) \times 1 = 1 + P_{data} \times (L_{data} - 1)$. In the gate-level optimization, the datapath divides into smaller signal paths $S_1, S_2, \dots, S_n$ with more specific forwarding and speculation on different netlist paths. The corresponding probability $P_1 + P_2 + \dots + P_n = P_{data}$, and the corresponding latency of these paths are no more than the longest path $\forall i \in 1, 2, \dots, n, L_i \leq L_{data}$. The CPI is calculated as:

$$CPI_{gate} = \sum_{i=1}^n P_i \times L_i + (1 - P_{data}) \times 1 \quad (1)$$

Thus $CPI_{gate} \leq \sum_{i=1}^n P_i \times L_{data} + (1 - P_{data}) \times 1 = CPI_{data}$. According to Equation 1, the throughput optimization is accomplished by reducing the gate-level latency of signal paths.

Dependency analysis in the vast gate-level design space is challenging and cannot be accomplished with conventional high-level human knowledge. We propose an efficient approach to resolve the RAW dependencies with gate-level forwarding and speculation. The latency decreases by assembling the logic gates of the short-delay paths in the fewer adjacent pipeline stages.

Fig. 2 shows how the latency of short-delay paths decreases with gate-level forwarding and speculation. Fig. 2(a) is a single datapath of a 3-stage multi-bit RAW. Since the module in the dataflow analysis omits all gate-level details, all bits in the data word share the datapath, and the only forwarding path are the red dashed lines (dashed lines represent the wire with no logic gates), assuming simultaneous arrivals of all signal bits. In Fig. 2(b), the multi-bit datapath is separated into several single-bit paths, and the latency of the short-delay paths can be reduced by forwarding. The dashed lines represent circuit wire without any logic gates. The illustrated multiplier example in Fig. 1 is an example of this kind of optimization, with the bit-level latency less than the multi-bit path latency. The bit-by-bit optimization can be applied by human designers case by case delicately, and with the same idea, we propose a further path-by-path gate-level analysis by forwarding and speculation.

In Fig. 2(c), the gate-level forwarding analyzes signal paths more concretely. A single-bit path can separate into various signal paths in the netlist because of the reuse signals or multiple fan-out logic gates, and the short-delay signal paths can be optimized with the same idea as well. We show an 8-path example in the figure, with each path valid for 1/8 possible inputs. The logic gates are represented with cloud shapes. Some of the paths (such as Path 1, the red path on the left) are long-delayed and have to be mapped into all three pipeline stages due to the clock period delay constraint, but

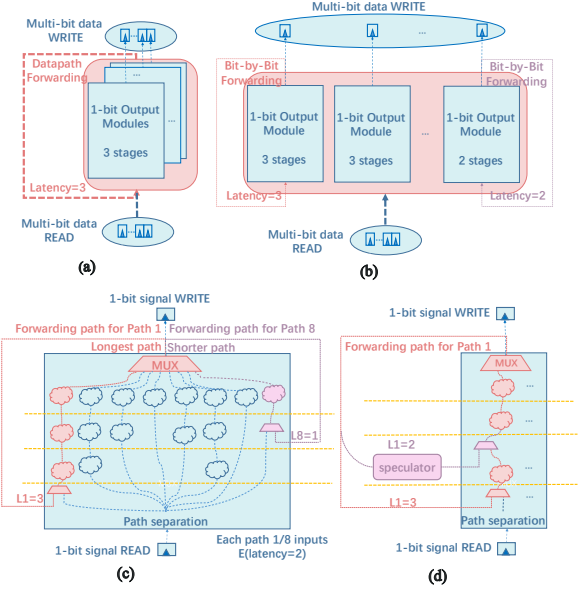


Figure 2: The overview of gate-level forwarding and speculation. (a) The RAW loop in the datapath with multi-bit data registers. (b) Forward by separating the datapath into bit-level paths and reducing short path latency. (c) Gate-level Forward by separating each bit-level bath based on signal paths in the netlist. Assemble logic gates in the short paths into adjacent pipeline stages, and thus, stages with no logic gates can be bypassed by gate-level forwarding. (d) Speculate to optimize latency on every forwarding path further.

the short-delay paths (such as Path 8, the purple path on the right) can be assembled in only one stage without any delay constraint violation. If all the logic gates of Path 8 are assembled in the last stage, the first two stages with no logic gates are only wires and can be bypassed by forwarding. The latency of Path 8 is reduced to 1. Thus, the expectation latency of all paths can be calculated as $E(L_{bit}) = \sum_{i=1}^8 P_i \times L_i = 2$.

In Fig. 2(d), a speculator can further optimize the signal path latency for short-delay paths. Even if few logic gates remains in some early stages after the assembling in the later stages, and are not entirely bypassed, these logic can be extracted in the speculator to predict the execution output before it is executed. These short-delay logic in the early stage does not need to consume an extra cycle to calculate but is assembled in the next pipeline stage with the speculator and further decreases the signal path latency.

Based on the efficient optimization based on gate-level forwarding and speculation, simple and direct algorithms are sufficient to design pipelines with better throughput than human designers. With more concrete heuristic algorithms for design space exploration, the design quality can be further improved, but it is not the primary consideration of this paper.

3.2 Gate-level Signal Forwarding

We propose a linear-complexity method to sufficiently find better gate-level forwarding than high-level human designs. First, the algorithm traverses the original netlist and numbers every gate

with the longest path from the gate to the output signals. Since the original circuit is single-cycle, the numbering can be done in linear complexity with a greedy traversal algorithm. Second, the gates are assembled to the latest possible pipeline stage one after another according to the numbers greedily, with the delay constraint of the pipeline design. Thus, the logic gates are denser for shorter paths in later stages. Even if some gates can be mapped to an early stage (because they can also be part of a long path), there is little logic in the earlier stages of the specific shorter path. With gate-level forwarding and speculation, the earlier stage of the specific shorter path can be bypassed and thus conceals the loop latency with gate-level analysis as shown in Figure 2. Thus, the pipeline stage boundaries are determined, and the gate-level forwarding paths can be inserted from the output signal to every earlier stage in advance, with a MUX to control whether each signal should be forwarded. For each signal forwarding path, if all the earlier stages before a specific pipeline stage are invalid, the forwarded signal is taken for all later pipeline stages to bypass all earlier stages.

3.3 Gate-level input speculation

Even if some earlier stages for the short paths are not entirely invalid, few logic gates remain; these pipeline stages can also be bypassed with a speculator containing the simple remaining logic.

First, we conduct software simulation to determine all inner signals in the execution, including whether the pipeline needs to stall. To simulate an n -stage pipeline, the simulator simulates the execution results of the following n cycles and determines whether errors are introduced without pipeline stalls. Only if the pipeline design calculates the different result as the input single-cycle circuit does the pipeline need to stall and wait for the RAW signal to write back. Thus, in theory, the simulator eliminates all redundant pipeline stalls. Unfortunately, the simulator cannot be represented as hardware since it uses future information by simulation.

Then, We use a Decision-Tree (DT) based program-by-example method to approximately fit the simulator with the valid signals, including the pipeline input signals and the fast-forwarded signals. The speculator is preliminarily programmed by a DT-based method, as mentioned in the present papers[24], and can be formally verified with an SMT solver to guarantee its functional correctness. Although the speculator is designed based on an approximate algorithm, it does not require the extra area/throughput overhead for error correction if it passes the formal verification. If the preliminary design cannot pass the verification, the PBE method samples more examples for more accurate designs. For formal verification, we unfold the pipeline design, and the single-cycle circuit can be represented in the Boolean expression for the SMT solver. The input single-cycle unpipelined circuit and the temporal unfolding circuit of the pipelined design can be formulated as C_{in} and C_{out} , correspondingly. The pipeline stall signal of the forwarding and the speculator controller is a single-cycle function and can be formulated as C_{stall} . The only possible error for the designed pipeline is that it does not stall with the forwarded or speculated input, but it should be stalled in the simulator. Otherwise, the pipeline execution result is different from the single-cycle circuit. Thus, the formal verification of the design can be formulated as $(C_{out} \equiv C_{in}) \wedge (C_{stall} = 0)$ and verified by the SMT solver.

4 EVALUATION

The proposed method is evaluated on widely used EDA benchmarks of varying sizes, including ISCAS 85 and EPFL circuits, and a custom real-world RISC-V-CPU benchmark. The RISC-V-CPU benchmark consists of the complete RISC-V-32I CPU design and partial components of the entire CPU system. Specifically, the benchmark includes a 32-bit INT adder, a 32-bit INT multiplier, a 32-bit FP adder, a 32-bit FP multiplier, the program counter (PC), a multi-function arithmetic logic unit (ALU), and a complete RISC-V-32I CPU. Circuit size is measured by the number of nodes in the And-Inverter Graph (AIG) representation compiled by Yosys, ranging from thousands to over 230,000 nodes. The average benchmark size is 14,401 AIG nodes, while the most complex circuit, EPFL-hyp, contains 232,134 AIG nodes. The functional circuits in the benchmark are integrated into real-world dataflows for typical RAW circumstances.

Our pipelining method runs on CentOS 7.8 with 2 Intel Xeon Gold 6230 CPUs and 512G Memory. We use the Gimsatol [7] as the SMT solver to formally validate the design. The software in the ASIC experiment is based on the 65nm library. The experiment performance is measured by the circuit throughput, time per inputs (TPI), or cycle per inputs (CPI). The circuit throughput is calculated by random 10,000-cycle dataflow on the pipeline.

4.1 Gate-level RAW optimization Necessity

Table 1 reports the area and throughput of the same pipeline structure with both high-level analysis and gate-level analysis. We compare the pipeline throughput of the best high-level pipeline design (*Human*) and the best throughput gate-level design in theory according to the simulator (*Simu.*). The result shows that gate-level dependency analysis provides more opportunities for pipeline throughput optimization. In 1/11 cases in ISCAS and 7/20 cases in EPFL, the high-level dependency analysis is not efficient enough in theory ($> 1\%$ throughput loss than *Simu.*) with 41% potential throughput optimization on average. In other cases, the gate-level design has the same throughput as the datapath-level design. We further evaluate the function on multi-function circuits closer to the real-world design scenario. With another functional netlist paralleled with the benchmark circuits, more diversities exist between long-delay and short-delay paths. The results show more gate-level optimization potentials in multi-function netlists than in single-function circuits. In all 11 cases in ISCAS and 19/20 cases in EPFL, the datapath-level design is not efficient enough in theory ($> 1\%$ throughput loss than *Simu.*) with 14% potential throughput optimization on average.

Unfortunately, the best-in-theory gate-level pipeline *Simu.* is unrealistic because it uses the exact execution result from the software simulator to decide whether the pipeline has to stall.

4.2 Forwarding/Speculation Efficiency

We evaluate the efficiency of our proposed method to exploit these potential gate-level optimizations. Table 1 reports the area and CPI of the proposed method with gate-level forwarding and speculation (*Ours*), and the optimization rate (Rate) comparing to the best-in-theory design (*Simu.*), showing the proposed method is efficient to increase the pipeline throughput with limit area overhead. In the single-function cases, the proposed method can eliminate over half of the possible redundant pipeline stalls. With only 3.6% extra area cost, the pipeline average throughput is optimized by 10%.

Table 1: The throughput optimization and area overhead of gate-level RAW optimization

single-function cases						
case	Human		Ours		Simu.	Rate
	area	CPI	area	CPI	CPI	
ISCAS/c2670	7972	1.67	8570	1.56	1.53	77.6%
EPFL/bar	11740	2.41	13038	2.28	2.27	96.3%
EPFL/div	67153	2.07	67153	2.07	1.03	0.00%
EPFL/i2c	8685	2.42	9989	1.89	1.87	99.8%
EPFL/int2float	1566	1.62	1605	1.53	1.47	60.3%
EPFL/priority	2954	2.00	2954	2.00	1.00	0.00%
EPFL/router	5659	1.45	5511	1.00	1.00	100%
EPFL/sqrt	32105	2.38	32139	2.38	1.98	0.00%
EPFL/voter	16499	1.97	16562	1.74	1.04	20.1%
Average	100%	100%	103.6%	90.2%	71.0%	50.45%
Average throughput	1×		1.10×		1.41×	
multi-function cases						
ISCAS/c1355	6775	1.98	6890	1.80	1.80	100%
ISCAS/c17	5868	1.98	5984	1.80	1.80	100%
ISCAS/c1908	6712	1.99	6826	1.81	1.81	100%
ISCAS/c2670	8917	2.38	13327	2.13	2.13	99.3%
ISCAS/c3540	7608	1.98	7722	1.80	1.80	100%
ISCAS/c432	6364	1.98	6460	1.80	1.80	100%
ISCAS/c499	6791	1.99	6906	1.80	1.80	100%
ISCAS/c5315	9680	2.38	11327	1.95	1.95	98.8%
ISCAS/c6288	8844	1.99	8958	1.81	1.81	100%
ISCAS/c7552	10713	2.40	11900	2.19	2.19	100%
ISCAS/c880	6901	1.99	7015	1.81	1.81	100%
EPFL/adder	9614	2.40	10774	2.18	2.18	100%
EPFL/arbitrator	23899	2.39	25062	2.17	2.17	99.9%
EPFL/bar	11262	2.39	12650	2.19	2.13	77.0%
EPFL/cavlc	7001	1.98	7134	1.79	1.79	100%
EPFL/ctrl	6233	1.99	6402	1.81	1.81	100%
EPFL/dec	8151	2.39	8764	2.12	1.87	61.4%
EPFL/div	68295	2.40	68295	2.40	1.63	0.00%
EPFL/i2c	10023	2.40	13327	2.10	2.10	100%
EPFL/int2float	6340	1.99	6458	1.82	1.82	100%
EPFL/log2	46296	1.98	46407	1.62	1.62	100%
EPFL/max	12729	2.40	12841	2.34	2.15	19.9%
EPFL/multiplier	37073	2.40	38236	2.19	2.19	100%
EPFL/priority	7164	1.97	7164	1.97	1.65	0.00%
EPFL/router	6445	2.00	6445	2.00	1.54	0.00%
EPFL/sin	14016	1.98	14180	1.80	1.80	99.8%
EPFL/sqrt	32750	2.41	32824	2.39	2.00	5.51%
EPFL/square	29948	2.40	31180	2.18	2.18	100%
EPFL/voter	22185	1.99	22185	1.99	1.78	0.00%
EPFL/hyp	243675	2.41	244835	2.20	2.20	100%
Average	100%	100%	105.5%	91.7%	87.7%	78.7%
Average throughput	1×		1.09×		1.14×	

One of these cases increases 45% throughput, with no additional area cost. The proposed method is also more efficient in multi-function cases, with diversities between long/short-delay paths and more redundant bubbles. The proposed method eliminates 78.7% redundant bubbles. With only 5.5% extra area cost, the pipeline average throughput is optimized by 9%.

4.3 Automatic pipeline Evaluation

We evaluate the overall design optimization of the automatic pipeline. With the same single-cycle unpipelined circuit as input, compare the throughput optimization by our methods, human designers, and EDA optimizations such as “retiming”. Theoretically, all the automatic pipelining methods cannot find more unnecessary bubbles than human designers based on their human-knowledge-based code analysis at the datapath level. Thus, the throughput of these methods cannot be better than the human design.

We illustrate the experiment result in Figure 3, showing the overall optimization of the pipeline designs in the EDA flow, human-design pipelines, and the proposed gate-level forwarding and speculation. The experiment results show that gate-level dependency

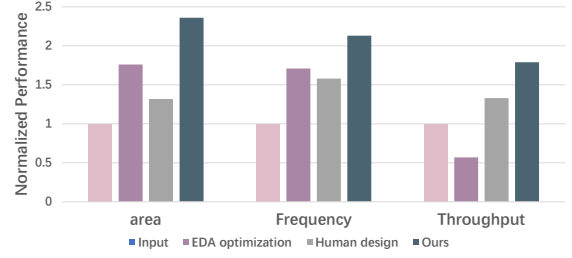


Figure 3: The average pipeline performance. The average pipeline performance of the pipeline optimization in the EDA flow, the human design, and our proposed method. The performance results are normalized with the input single-cycle unpipelined circuit, and we report the GEOMEAN of area, frequency, and throughput of the pipeline designs.

graph optimization is more efficient than the other automatic pipeline optimizations and finds better pipelines than human designers in some cases. The proposed method gets the pipeline with the highest frequency (1.87×) and the highest average throughput (1.57×). In 25/38 cases, the proposed automatic pipeline gets the best throughput, while human-designed pipelines get the best throughput /in the other 13/38 cases.

5 CONCLUSION

We propose a pipeline optimization based on automatic gate-level dependency analysis. Since human designers cannot explore the expanded vast gate-level design space, the proposed automatic machine resolves the challenge with an efficient methodology based on gate-level forwarding and speculation. The pipelined circuits conceal the loop latency of short-delay signal paths and eliminate redundant pipeline stalls. The results show that automatic pipelining can get a better pipeline design using the proposed optimization than human experts. The future work is to optimize the efficiency of the methodology further and apply it in the current EDA flow.

6 ACKNOWLEDGEMENTS

This work is partially supported by the NSF of China (under Grants 62222214, U22A2028, 62102399, 62372436, 62302483), CAS Project for Young Scientists in Basic Research (YSBR-029), Youth Innovation Promotion Association CAS.

REFERENCES

- [1] Mythri Alle, Antoine Morvan, and Steven Derrien. 2013. Runtime dependency analysis for loop pipelining in high-level synthesis. In *Proceedings of the DAC*. 1–10.
- [2] Pei-Wei Chen, Yu-Ching Huang, Cheng-Lin Lee, and Jie-Hong Roland Jiang. 2020. Circuit learning for logic regression on high dimensional boolean space. In *Proceedings of the DAC*. IEEE, 1–6.
- [3] Jason Cong, Yiping Fan, and Zhiru Zhang. 2004. Architecture-level synthesis for automatic interconnect pipelining. In *Proceedings of the DAC*. 602–607.
- [4] Jason Cong and Chang Wu. 1997. FPGA synthesis with retiming and pipelining for clock period minimization of sequential circuits. In *Proceedings of the DAC*. 644–649.
- [5] Jordi Cortadella, Marc Galceran-Oms, and Mike Kishinevsky. 2010. Elastic systems. In *Proceedings of the MEMOCODE*. IEEE, 149–158.
- [6] Steven Derrien, Thibaut Marty, Simon Rokicki, and Tomofumi Yuki. 2020. Toward speculative loop pipelining for high-level synthesis. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 39, 11 (2020), 4229–4239.
- [7] Mathias Fleury and Armin Biere. 2022. Scalable Proof Producing Multi-Threaded SAT Solving with Gimsat through Sharing instead of Copying Clauses. *ArXiv*

Table 2: The automatic pipelining result

case	Input Circuit			Ours			Human Design			EDA Optimization		
	area	clock	TPI	area	clock	TPI	area	clock	TPI	area	clock	TPI
ISCAS/c1355	30591.36	0.81	0.81	75727.08	0.34	0.36	24339.96	0.57	0.62	55265.40	0.37	1.11
ISCAS/c17	2372.04	0.40	0.40	6548.40	0.38	0.41	3181.32	0.36	0.42	3964.32	0.30	0.90
ISCAS/c1908	18661.68	0.99	0.99	52651.08	0.66	0.70	27500.76	0.66	0.72	46533.24	0.43	1.29
ISCAS/c2670	25059.24	0.57	0.57	92535.48	0.39	0.48	46449.72	0.41	0.54	67435.92	0.32	0.96
ISCAS/c3540	33405.12	1.10	1.10	57533.76	0.75	0.86	28362.96	0.77	0.90	45243.00	0.42	1.26
ISCAS/c432	19094.40	0.76	0.76	18745.20	0.56	0.63	10472.76	0.47	0.55	16070.04	0.36	1.08
ISCAS/c499	30656.52	0.80	0.80	72106.20	0.65	0.69	35790.12	0.59	0.64	54335.16	0.36	1.08
ISCAS/c5315	24847.56	0.73	0.73	98941.68	0.34	0.43	43602.12	0.50	0.66	65770.92	0.37	1.11
ISCAS/c6288	26831.52	2.31	2.31	80499.60	1.12	1.23	46073.52	0.88	0.96	49871.16	0.79	2.37
ISCAS/c7552	30279.24	0.91	0.91	81793.00	0.34	0.42	44851.68	0.60	0.79	67880.16	0.43	1.29
ISCAS/c880	20404.44	0.75	0.75	67594.30	0.61	0.69	33892.56	0.48	0.56	63852.10	0.60	1.80
EPFL/adder	36338.05	0.84	0.84	97656.12	0.70	0.93	55373.04	0.50	0.66	74336.04	0.84	1.11
EPFL/arbitrator	49169.16	0.78	0.78	94766.76	0.46	0.60	47595.96	0.68	0.90	96584.76	0.64	1.92
EPFL/bar	38379.24	0.81	0.81	112939.56	0.55	0.67	35515.08	0.64	0.85	71096.76	0.74	2.22
EPFL/cavlc	10676.88	0.66	0.66	30485.52	0.45	0.48	17188.56	0.39	0.43	23349.24	0.32	0.96
EPFL/ctrl	18843.12	0.54	0.54	67589.28	0.51	0.54	30732.48	0.29	0.32	44133.48	0.47	1.41
EPFL/dec	29949.84	0.53	0.53	79152.12	0.49	0.54	44798.04	0.34	0.40	58439.16	0.51	1.53
EPFL/div	230412.96	28.53	28.53	279177.12	8.40	11.07	237796.20	18.95	25.05	311493.60	20.02	60.06
EPFL/i2c	36072.00	0.62	0.62	83765.16	0.37	0.46	58780.44	0.39	0.52	66336.48	0.61	1.83
EPFL/int2float	6956.28	0.58	0.58	19323.72	0.39	0.42	10004.76	0.38	0.41	14091.48	0.53	1.59
EPFL/log2	97319.88	6.66	6.66	104479.20	2.56	2.80	128829.60	5.12	5.58	123654.96	6.80	20.40
EPFL/max	36579.60	1.23	1.23	91648.08	0.35	0.46	43211.16	0.81	1.07	72939.96	0.46	1.38
EPFL/mem-ctrl	100884.24	1.41	1.41	186355.80	0.42	0.42	133359.12	0.53	0.70	162291.96	1.12	3.36
EPFL/multiplier	91193.04	3.62	3.62	97772.40	1.68	2.26	104988.60	2.65	3.50	135907.92	1.06	3.18
EPFL/priority	9122.40	0.62	0.62	23535.36	0.57	0.67	15890.76	0.35	0.41	19546.20	0.58	1.74
EPFL/router	22835.16	0.60	0.60	115236.00	0.57	0.57	48775.68	0.39	0.46	53935.20	0.35	1.05
EPFL/sin	32697.00	3.33	3.33	65064.96	1.28	1.40	43946.28	1.47	1.60	55254.24	1.07	3.21
EPFL/sqrt	85061.52	104.82	104.82	106145.64	39.89	52.56	54670.32	134.25	177.46	115683.12	104.23	312.69
EPFL/square	73950.48	1.82	1.82	97045.20	1.04	1.38	87083.64	1.22	1.61	126078.84	0.60	1.80
EPFL/voter	16610.40	1.56	1.56	22163.40	0.77	0.90	32972.76	1.11	1.30	24344.28	0.61	1.83
EPFL/hyp	433728.00	567.32	567.32	777208.70	67.23	88.72	533431.10	215.32	284.62	483114.00	525.12	525.13
RISCv/adder-int	31239.36	0.73	0.73	75783.96	0.36	0.40	34701.48	0.44	0.52	49371.84	0.65	1.95
RISCv/adder-fp	28475.64	1.31	1.31	76090.67	0.73	0.84	39900.24	0.90	1.06	51772.04	0.91	2.73
RISCv/multi-int	38120.76	1.55	1.55	83439.72	0.37	0.43	41266.44	0.89	1.04	62889.48	1.30	3.90
RISCv/multi-fp	34030.80	1.51	1.51	76970.52	0.40	0.47	29722.32	0.82	0.96	58255.92	1.26	3.78
RISCv/pc	6823.80	0.68	2.04	9861.84	0.49	1.47	9789.48	0.50	1.50	9789.48	0.50	1.50
RISCv/alu	49692.60	1.64	1.64	100269.72	1.07	1.25	39579.12	1.21	1.42	78219.00	1.43	4.29
RISCv/RISCv-32I	34489.44	0.74	0.74	86963.76	0.54	0.61	56076.48	0.45	0.53	85821.12	0.34	1.02
Average	100%	100%	100%	221%	53.5%	63.8%	127%	64.4%	79.5%	180%	62.4%	187%
Average throughput	1×			1.57×			1.26×			0.53×		

abs/2207.13577 (2022).

- [8] Freddy Gabbay and Avi Mendelson. 1996. *Speculative execution based on value prediction*. Citeseer.
- [9] Marc Galceran-Oms, Jordi Cortadella, Dmitry Bufistov, and Mike Kishinevsky. 2010. Automatic microarchitectural pipelining. In *Proceedings of the DATE*. IEEE, 961–964.
- [10] Ilya Ganusov, Henri Fraisse, Aaron Ng, Rafael Trapani Possignolo, and Sabya Das. 2016. Automated extra pipeline analysis of applications mapped to Xilinx UltraScale+ FPGAs. In *Proceedings of the FPL*. IEEE, 1–10.
- [11] Sumit Gulwani, Oleksandr Polozov, Rishabh Singh, et al. 2017. Program synthesis. *Foundations and Trends® in Programming Languages* 4, 1-2 (2017), 1–119.
- [12] Ching-Yi Huang, Chi-An Rocky Wu, Tung-Yuan Lee, Chih-Jen Jacky Hsu, and Kei-Yong Khoo. 2019. 2019 CAD contest: Logic regression on high dimensional boolean space. In *Proceedings of the ICCAD*. IEEE, 1–6.
- [13] Matei Iştoan and Florent De Dinechin. 2017. Automating the pipeline of arithmetic datapaths. In *Proceedings of the DATE*. IEEE, 704–709.
- [14] Timothy Kam, Michael Kishinevsky, Jordi Cortadella, and Marc Galceran-Oms. 2008. Correct-by-construction microarchitectural pipelining. In *Proceedings of the ICCAD*. IEEE, 434–441.
- [15] Mikko H Lipasti and John Paul Shen. 1996. Exceeding the dataflow limit via value prediction. In *Proceedings of the MICRO*. IEEE, 226–237.
- [16] Junyi Liu, John Wickerson, Samuel Bayliss, and George A Constantinides. 2017. Polyhedral-based dynamic loop pipelining for high-level synthesis. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 37, 9 (2017), 1802–1815.
- [17] Maria-Cristina V Marinescu and Martin Rinard. 2001. High-level automatic pipelining for sequential circuits. In *Proceedings of the ISSS*. 215–220.
- [18] Somsubhra Mondal and Seda Ögrenci Memik. 2005. Resource sharing in pipelined CDFG synthesis. In *Proceedings of the ASPDAC*. 795–798.
- [19] Syed Waqar Nabi and Wim Vanderbauwhede. 2019. Automatic pipelining and vectorization of scientific code for FPGAs. *International Journal of Reconfigurable Computing* 2019 (2019), 1–12.
- [20] Asger Munk Nielsen, David W Matula, Chung Nan Lyu, and Guy Even. 2000. An IEEE compliant floating-point adder that conforms with the pipeline packet-forwarding paradigm. *IEEE Trans. Comput.* 49, 1 (2000), 33–47.
- [21] Eriko Nurvitadhi, James C Hoe, Timothy Kam, and Shih-Lien L Lu. 2011. Automatic pipelining from transactional datapath specifications. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 30, 3 (2011), 441–454.
- [22] Hossein Omidian and Guy GF Lemieux. 2019. Low-level Loop Analysis and Pipelining of Applications Mapped to Xilinx FPGAs. In *Proceedings of the FPL*. IEEE, 391–396.
- [23] Arthur Perais and André Sez nec. [n.d.]. Practical data value speculation for future high-end processors. In *Proceedings of the HPCA, pages=428–439, year=2014, organization=IEEE*.
- [24] Shubham Rai, Walter Lau Neto, Yukio Miyasaka, Xinpei Zhang, Mingfei Yu, Qingyang Yi, Masahiro Fujita, Guilherme B Manske, Mathews F Pontes, Leomar S da Rosa, et al. 2021. Logic synthesis meets machine learning: Trading exactness for generalization. In *Proceedings of the DATE*. IEEE, 1026–1031.
- [25] Chittoor V Ramamoorthy and Hon Fung Li. 1977. Pipeline architecture. *Comput. Surveys* 9, 1 (1977), 61–102.
- [26] John Paul Shen and Mikko H Lipasti. 2013. *Modern processor design: fundamentals of superscalar processors*. Waveland Press. 82 pages.
- [27] Yuzhe Tang and Bugra Gedik. 2012. Autopipelining for data stream processing. *IEEE Transactions on Parallel and Distributed Systems* 24, 12 (2012), 2344–2354.